

S³ Gym：大语言模型能否将自我测试与自我评判转化为自我提升？

1 字节跳动种子团队，2M-A-P，3TokenWave.AI

完整作者列表见贡献部分

Abstract

大语言模型（Large Language Model）日益频繁地与外部环境交互并积累大量行为经验，然而现有智能体基准大多将其作为固定策略进行评估。因此，智能体能否主动测试自身行为、评判所得经验，并利用这些经验改进未来决策，仍是一个未解问题。本文提出 **S³ Gym**，一个通过三项耦合能力评估大语言模型自我提升的交互式基准：

自我测试、自我评判与自我提升。**S³ Gym** 将宽松探索与严格的留出评估相分离，并在七个基于文本的游戏中实例化该协议，每个游戏均配备可执行的环境验证器。本文评估了三种纳入交互经验的途径：直接历史上下文学习、基于评分的摘要记忆以及参数训练。实验结果表明，自我提升既非自动发生，也非均匀一致。上下文层面的经验在若干模型-游戏组合中提升了性能，但最有效的途径高度依赖于任务结构：当经验可被压缩为可复用的策略规则时，摘要具有优势；而当成功依赖于精确的、状态相关的信息时，摘要往往不如原始历史。参数训练在部分任务上带来显著提升，但在其他任务上表现出不稳定的改进和严重的负迁移。这些发现表明，识别成功动作并不足够；智能体还必须将反馈转化为可执行且可迁移的策略。**S³ Gym** 为诊断这一过程并识别阻碍智能体将交互经验转化为可靠自我提升的瓶颈提供了统一框架。

Date: September 1, 2026

Project Page: <https://self-developing-agents.github.io/>

1 引言

大语言模型（Large Language Model）日益被部署为自主智能体，它们调用工具、执行代码，并通过反复的观察—推理—行动—反馈循环与外部环境交互。这种交互是自动化研究、软件工程和开放式决策等应用的核心，在这些应用中，智能体持续积累行为经验。现有基准评估多轮决策、环境探索和任务完成 [15, 16]，但它们通常将被评估的模型视为固定策略。因此，它们主要回答一个静态问题——模型在评估时的能力如何？——而对于模型能否利用自身过去的交互来改进未来行为，提供的洞见有限。我们将这种由经验驱动的能力称为自我改进。

经验学习与反思性探究理论表明，经验只有在以下情况下才变得有用：

它被积极地检验、抽象化并再次测试。杜威和科尔布强调通过反思和再应用将经验转化为知识 [8, 14]，而波普尔将进步描述为将猜想反复暴露于能够揭示错误的测试中 [21]。同样的原则适用于大语言模型智能体：仅仅收集轨迹并不能保证学习。智能体必须生成信息丰富的证据，解释成功与失败的原因，并将这些解释转化为改进的行为。因此，我们将经验驱动的学习分解为三种相互依存的能力：自我测试，即智能体探索策略并收集诊断证据；自我判断，即评估行动、结果及其潜在的可重用性；以及自我改进，即由此产生的经验改变未来的决策 [2, 23]。

经过测试和判断的经验可以在不同持久性水平上被纳入。在短期上下文学习层面，智能体直接根据先前的观察、行动和得分来调整未来的决策。在中期外部记忆层面，长轨迹被压缩为自然语言规则、失败模式或可复用技能，如反思（Reflexion）和航海者（Voyager）[26, 30]。在长期训练层面，经过筛选或修正的轨迹通过参数更新被内化，遵循由自举推理（STaR）、强化自训练（ReST）和重复强化自训练（Re-ReST）[9, 11, 37] 所探索的更广泛方向。这些机制做出了不同的权衡：原始历史保留了详细证据但消耗上下文，摘要提供了紧凑的抽象但依赖于判断质量，而训练提供了持久的内化但也可能放大被错误判断的经验。因此，需要一个统一的评估协议，在相同的交互和测试条件下比较这些路径。

为了满足这一需求，我们引入了 **S³ 健身房（Gym）**，全称为自测试、自判断与自改进健身房，这是一个交互式基准，用于评估大语言模型（LLM）能否通过自身的环境经验变得更加有能力。每次评估包括一个探索阶段和一个评估阶段，反映了搜索新策略与利用已获得知识之间的区别 [17]。在探索期间，智能体与相对宽松的环境配置进行交互，生成成功、不成功和部分成功的轨迹。重要的是，S³ 健身房明确地将模型生成的自判断与环境可验证的结果分开：自判断决定了交互经验如何被组织和复用，而环境得分衡量了由此产生的行为性能。更新后的智能体随后在更严格且留出的配置上进行评估，直接测试探索期间获得的经验是否能迁移到新条件，以及自判断中的错误是否成为后续自改进的瓶颈。

我们以文本游戏为基础实例化 S³ Gym（图 1），遵循 KORgym 的游戏化评估范式 [25]。KORgym 通过多样化的交互游戏评估大语言模型（LLM）的推理能力，而 S³ Gym 则将这些环境用作受控测试平台，用于经验驱动的自我改进。其多轮交互、部分可观测性、延迟后果以及长时程决策提供了丰富的行为轨迹，同时程序化可验证的规则使得可复现的评估成为可能，并具有客观的步骤级或终局得分。相关环境，包括 TextWorld、ALFWorld 和 ScienceWorld，同样展示了文本交互在评估规划、具身推理和科学问题解决方面的实用性 [5, 27, 31]。S³ Gym 包含七款游戏——国际象棋、扫雷、消除、俄罗斯方块、贪吃蛇、植物大战僵尸和信任演化——涵盖潜在规则归纳、约束满足、数值变换、空间规划、资源分配、生存以及多智能体策略。每款游戏提供相关但不同的探索与评估配置，要求智能体从经验中提取可迁移的知识，而非记忆孤立的动作或种子。

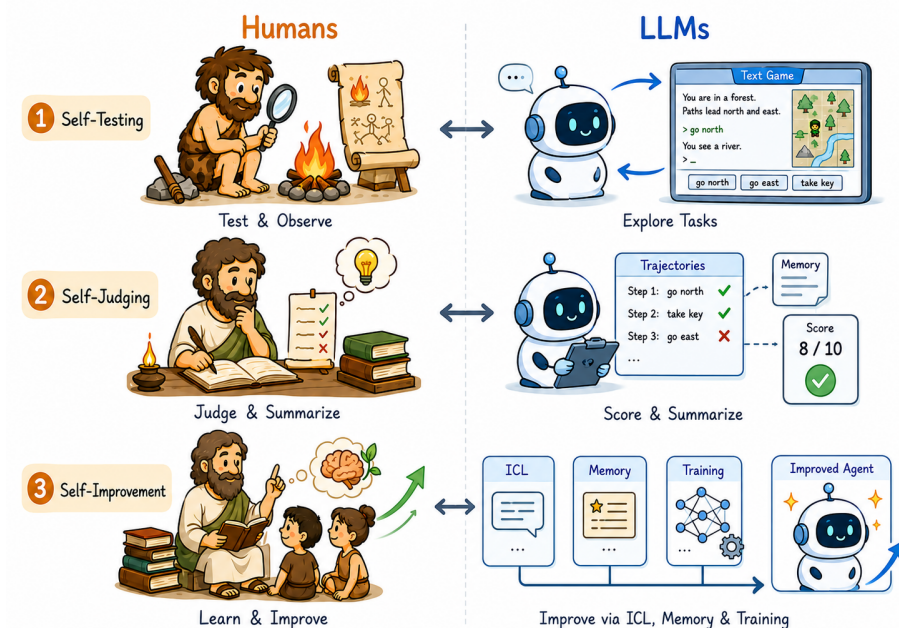


图 1 人类与大语言模型智能体经验驱动改进的概念性概述。两个过程都涉及通过自我测试生成经验，通过自我判断解读经验，并通过可复用的记忆或学习机制将其纳入。

本文的主要贡献总结如下：

- **经验驱动自我改进的统一表述。**我们将大语言模型自我改进形式化为一个迭代过程，连接自我测试、自我判断以及随后的行为改变，并引入 S₃ Gym，具有独立的探索阶段和留出评估阶段。
- **经验整合路径的系统比较。**我们在统一协议下评估了三种纳入交互经验的机制——历史上下文学习、摘要记忆和参数训练。
- **自我判断的显式评估。**通过比较模型生成的判断与验证器计算的结果，本文评估智能体能否识别有用和有害的经验，并将这些判断转化为可操作的改进方向。
- **全面的实证与诊断分析。**本文在七个交互式环境中评估代表性大语言模型，并分析改进动态、判断可靠性、摘要有效性、跨游戏一致性以及常见失败模式。

2 背景

大多数智能体基准在问题已经可执行之后才开始：任务已指定，奖励或评判器已定义，执行脚手架已固定。这种设置对于受控比较是必要的，但它掩盖了在实际部署中占主导地位的工作。在工业界，这些工作分布在多个角色之间——包括解决方案架构师、应用工程师和平台工程师——但其最近最明显的结晶是前向部署工程师（Forward-Deployed Engineer, FDE），这一头衔由 Palantir 推广，并已被前沿模型公司采用 [18, 20]。前向部署工程师嵌入部署环境中，将通用模型转变为能够与客户的数据格式、工作流程和操作约束协同工作的系统。该角色的成功标准不是演示或基准分数，而是部署的系统是否真正被使用、持续运行，并在面对失败时不断改进。

前向部署工程师角色在前沿模型和数据平台公司中的增长反映了一个简单事实：一个有能力的模型还不是一个可工作的系统 [3, 29], 而如今这一差距主要由人类工程师来弥补。

从模型侧来看, FDE 工作提供了基准设计者通常预设的三部分结构。首先, 目标模糊: 非正式的部署需求必须转化为具体的目标、约束和成功标准。其次, 反馈信号可能缺失或不可靠: 在任何人能够判断系统是否在改进之前, 必须构建测试、评判器、迹或其他验证机制。第三, 执行系统可能尚不存在可用形式: 未来任务运行所需的工具、上下文管理、状态、生命周期逻辑和验证接口, 必须随着需求变化而构建、适配和维护。S³ Gym 聚焦于第二层: 面向智能体的反馈信号及其所支持的、由经验驱动的改进循环。该基准保留可执行环境验证器作为外部真实依据, 但在探索过程中对智能体隐藏验证器计算出的结果。智能体必须通过自测试生成信息性证据, 通过自评判评估该证据, 并通过自改进将其转化为更好的未来行为。这种分离既揭示了自评判是否与实际结果一致, 也揭示了经过评判的经验是否产生可迁移的改进。Aspire 研究广泛的部署需求如何转化为能力增长, HarnessDev 研究模型如何构建和维护承载这些能力的执行系统, 而 S³ Gym 则隔离了可执行环境中的经验到改进循环。

3 相关工作

3.1 交互式智能体基准

大语言模型智能体的最新进展推动了交互式基准的发展, 这些基准评估多轮决策、工具使用、规划和环境交互。AgentBench [15] 在操作系统、数据库和游戏等多个领域评估大语言模型智能体, 而 AgentBoard [16] 则对多步任务中的智能体进展和失败进行细粒度分析。基于文本的环境如 TextWorld [5]、ALFWorld [27] 和 ScienceWorld [31] 进一步为语言引导的规划、具身推理和科学问题求解提供了可控的测试平台。近期面向游戏的基准也在文本或可编程验证的环境中考察交互式推理和策略行为。

这些基准为评估智能体行为提供了丰富的素材, 但它们通常将被评估的模型视为固定策略。交互轨迹主要用于在预定义的评估设置中衡量任务完成情况或诊断失败原因, 而非判断智能体能否将自身经验转化为改进的未来行为。S³ Gym 建立在交互环境的可控性和客观验证之上, 同时将评估目标从静态任务能力转向经验驱动的行为改进。

3.2 基于经验的智能体改进

越来越多的研究关注语言智能体如何通过复用自身交互经验来改进。在上下文层面, Reflexion [26] 引入了言语强化学习, 智能体反思先前的尝试, 并利用文本反馈指导未来决策。在外部记忆层面, Voyager [30] 在不断演化的库中积累可复用技能, 使具身智能体能够保留并复用开放式交互中获得的知识。近期基于记忆的方法同样将轨迹压缩为规则、失败模式或可复用策略, 供后续情节检索: ReasoningBank [19] 从智能体自我判断的成功与失败经验中提炼可泛化的推理策略, Treeof-Experience [7] 将积累的经验组织为层级结构, 支持反馈归因与跨任务迁移, MetaSkill-Evolve [33] 通过双时间尺度的元技能演化, 将技能文件重写扩展到改进过程本身。

经验也可以通过参数更新来纳入。STaR [37]、ReST [11] 和 Re-ReST [9] 通过生成、筛选或精炼自产生的推理轨迹来改进语言模型。Self-Rewarding Language Models [36] 进一步利用模型生成的偏好信号进行迭代优化。

在交互式环境中，RetroAgent [39] 利用回顾性双重内在反馈训练智能体，使跨回合获取的经验能够被检索和复用，而非仅隐式编码于参数中；测试时自改进 [1] 则从智能体自身的交互失败中构建针对性训练样本。这些研究表明，交互历史与自生成监督能够提升模型性能。然而，它们通常仅在特定任务或训练设置下考察单一改进机制，难以厘清改进或失败究竟源于所收集经验的质量、经验解释的可靠性，还是用于整合该经验的机制。S³ Gym 提供了一个统一的评估协议，在相同的探索与留出评估设置下，比较历史上下文学习、摘要记忆和参数训练等不同经验整合策略。

3.3 评估自适应与自改进智能体

模型生成监督的可靠性已在基于大语言模型的评估研究中得到广泛探讨。大模型作为评审 [41] 与 JudgeBench [28] 表明，语言模型能够提供有用的评估信号，但其判断仍易受偏差、校准误差和推理局限性的影响。近期一项针对 ALFWorld 中步骤级信用分配的审计进一步显示，在与执行回放对比时，大模型评审分数、结果条件对数概率比以及策略自身置信度均未能以优于随机水平识别因果重要步骤 [38]。此类局限性对自改进智能体尤为重要，因为不准确的判断可能导致有害经验被保留，或有价值的经验被丢弃。

近期基准测试已开始从静态能力评估转向自适应与自我改进系统。PostTrainBench [22] 评估在留出任务上改进模型参数的自动化后训练流程。SEA-Eval [13] 通过顺序任务交互研究长期智能体演化，而 SEAGym [40] 则考察持久智能体框架（包括提示、记忆、工具与工作流）的演化。PAST-Bench [34] 通过开启与关闭保留经验来运行匹配任务序列，检验保留经验是否真正改善个人智能体。ContinualSkillBench [10] 测量上下文中的持续技能学习，并发现不同模型与领域间的增益差异显著。FinEvo-Bench [6] 为专业金融工作流中的自我演化智能体提供纵向基准。Social Gym [12] 在具有规则可验证结果的游戏基准测试多智能体社会推理。AI4AI-Bench [4] 隔离了用于递归自我改进的训练算法设计。除基准测试外，近期分析揭示自我改进本身往往不可靠：基于记忆的方法表现出高方差和对任务顺序的强敏感性 [35]；一旦有缺陷的技能进入技能池，累积技能可能污染后续行为 [24]；优化器驱动的智能体增益在持续学习轮次中未必复合增长 [32]。这些基准与分析是评估自适应系统的重要步骤，但它们关注不同的改进工件或适应过程的特定方面，且无一将自测试与可执行环境验证下的显式自判断相结合。表 1 总结了现有基准与 S³ Gym 之间的关键差异。

与主要评估演化结果或单一适应机制的现有基准不同，S³ Gym 明确研究智能体将交互经验转化为未来行为改进的过程。它在相同环境和交互预算内评估多种经验整合策略，同时在不相交且通常更严格的配置上测试更新后的智能体。此外，S³ Gym 同时记录模型生成的判断和可执行环境结果，从而能够分析改进失败是源于不准确的经验评估，还是源于无法将正确识别的反馈转化为可迁移行为。

表 1 自适应与自改进语言智能体基准的比较。

Benchmark	Updated System	Self-Testing	Self-Judging	Experience Use	Evaluation
PostTrainBench	Model	×	×	Parameter training	Held-out tasks
SEA-Eval	Agent	Partial	×	Sequential adaptation	Long-term evolution
SEAGym	Agent harness	Partial	×	Prompt, memory, tool, workflow updates	ID/OOD transfer
PAST-Bench	Agent memory	Partial	×	Retained cross-session experience	Matched on/off conditions
ContinualSkillBench	Skill library	Partial	×	In-context skill accumulation	Cross-task reuse
FinEvo-Bench	Agent	Partial	×	Longitudinal experience	Longitudinal task sequences
Social Gym	Agent	Partial	×	Tournament self-play	Rule-verifiable outcomes
S³Gym	Agent behavior	✓	✓	History ICL, Summary Memory, Training	Strict held-out evaluation

4 方法

4.1 概述

S³ Gym 通过在交互环境中测试策略、判断产生的经验并复用所获得的知识，来评估大语言模型（LLM）能否改进其未来行为。一个自改进循环包含五个阶段：

$$\mathcal{R}_t^{(p)} = \text{Explore} \rightarrow \text{Judge} \rightarrow \text{Consolidate} \rightarrow \text{Update}_p \rightarrow \text{Evaluate}, \quad (1)$$

其中 t 为循环索引， $p \in \{\text{历史}, \text{记忆}, \text{训练}\}$ 表示改进路径。在探索阶段，智能体在相对宽松的游戏配置下积累轨迹。随后，它判断自身的决策，并将产生的经验整合为原始历史、摘要记忆或训练样本。最后，更新后的智能体在更严格且留出的配置上进行评估。

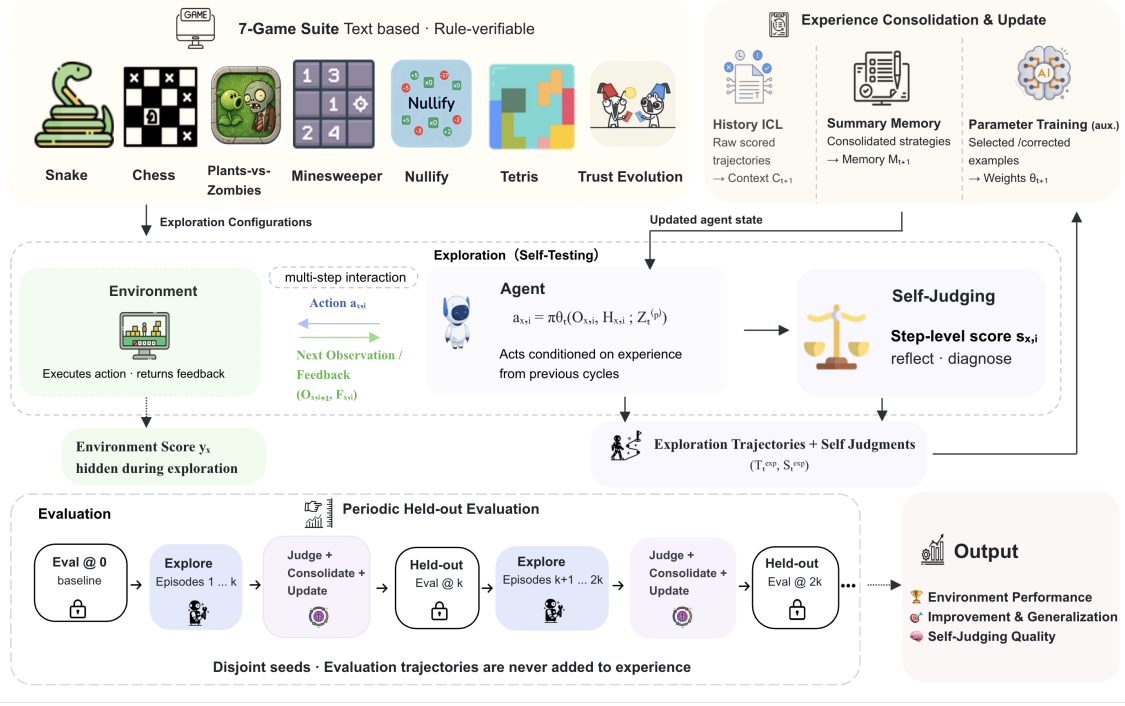


图 2 S 3 Gym 概述。探索轨迹经过判断后，通过历史上下文学习（ICL）、摘要记忆或参数训练进行整合。更新后的智能体随后在更严格的游戏配置上进行评估。

4.2 交互协议与形式化

每个游戏包含多个独立回合，每个回合由一系列交互步骤组成。对于第 x 个回合的第 i 步，我们记当前观测为 $O_{x,i}$ ，回合内交互历史为 $H_{x,i}$ ，智能体动作为 $a_{x,i}$ ，智能体自评的即时分数为 $s_{x,i}$ ，可观测的环境反馈为 $F_{x,i}$ ，验证器计算的步骤奖励为 $r_{x,i}$ ，最终环境分数为 y_x 。

一个回合被初始化为

$$O_{x,0} = \text{Reset}(\text{seed}_x, c_x), \quad (2)$$

$$H_{x,0} = \emptyset, \quad (3)$$

其中 c_x 表示游戏配置。在每一步，智能体接收当前观测、回合内历史以及从先前探索回合继承的通路特定经验状态 $Z_t^{(p)}$ 。遵循实际交互接口，模型联合产生一个动作和一个自我判断的即时分数：

$$(a_{x,i}, s_{x,i}) = \pi_{\theta_t} \left(O_{x,i}, H_{x,i}; Z_t^{(p)} \right), \quad (4)$$

其中 $p \in \{\text{历史}, \text{记忆}, \text{训练}\}$ 。分数 $s_{x,i}$ 表示模型对其在游戏规则下所提动作的即时奖励的估计，而非自由形式的置信度分数。在上下文级通路中， $Z_t^{(p)}$ 包含保留的原始历史或总结的经验；对于参数训练，获得的经验主要通过更新后的参数 θ_t 来体现。

环境随后执行所提动作，并返回下一观测、可观测反馈、验证器计算的奖励以及终止信号：

$$(O_{x,i+1}, F_{x,i}, r_{x,i}, d_{x,i}) = \text{Env}(O_{x,i}, a_{x,i}). \quad (5)$$

重要的是， $F_{x,i}$ 表示可供智能体用于后续交互的环境信息，而 $r_{x,i}$ 由基准作为外部验证信号保留。在主设置中，验证器计算的奖励在探索期间不暴露给智能体。这种分离使得 S³ Gym 能够将智能体的自我判断 $s_{x,i}$ 与实际环境奖励 $r_{x,i}$ 进行比较，而不允许真实奖励直接指导后续决策。

智能体可见的转移随后被迫加到回合内历史中：

$$H_{x,i+1} = H_{x,i} \oplus \text{Format}(O_{x,i}, a_{x,i}, s_{x,i}, F_{x,i}, O_{x,i+1}), \quad (6)$$

其中 $\oplus$ 表示有序拼接。因此，交互历史记录智能体自身的分数以及状态转移，而验证器奖励 $r_{x,i}$ 仍为基准侧信息。在下一步，智能体再次将公式 (4) 应用于更新后的观测和历史。

当回合终止或达到最大步数时，环境返回验证器计算的最终分数

$$y_x = \text{Score}(\tau_x), \quad (7)$$

其中 τ_x 表示回合 x 的完整执行轨迹。步级奖励 $r_{x,i}$ 用于评估自我评判的可靠性，而 y_x 衡量用于基准评估的回合级行为性能。在主设置中，探索期间智能体不会获得这两种信号。

4.3 自我评判与经验整合

当智能体无法获得精确的环境奖励时，自我评判提供了一种内部信号，用于识别有用和有害的经验。低分轨迹段表明可能存在错误或应避免的策略，而高分轨迹段则提供了可保留的候选策略。S³ Gym 以两种上下文级形式组织评判后的经验。

历史上下文。带分数标注的轨迹被直接序列化并附加到未来的上下文中：

$$C_{t+1} = \text{Serialize}(C_t, \mathcal{T}_t^{\text{exp}}, \mathcal{S}_t^{\text{exp}}), \quad (8)$$

其中 $\mathcal{T}_t^{\text{exp}}$ 和 $\mathcal{S}_t^{\text{exp}}$ 分别是第 t 轮中的探索轨迹及其自我评判分数。

摘要记忆。模型将评判后的轨迹压缩为可复用的经验：

$$M_{t+1} = \text{Summarize}(M_t, \mathcal{T}_t^{\text{exp}}, \mathcal{S}_t^{\text{exp}}). \quad (9)$$

摘要包含需要保留的策略、需要避免的错误以及下一轮交互的具体方向：

$$M_{t+1} = (R_t^{\text{retain}}, R_t^{\text{avoid}}, D_t^{\text{next}}). \quad (10)$$

4.4 探索与评估阶段

每个游戏提供一个探索分布 C_a^{exp} 和一个评估分布 C_a^{eval} 。探索环境保留主要任务结构，但采用更宽松的条件，例如更小的状态空间、更弱的失败惩罚或额外的尝试机会。评估环境则使用更严格的规则或更复杂的配置。

一个自我改进循环包含 N_{exp} 个探索回合，随后是 N_{eval} 个评估回合：

$$\{\tau_{t,x}^{\text{exp}}\}_{x=1}^{N_{\text{exp}}} \rightarrow \text{Improve} \rightarrow \{\tau_{t,x}^{\text{eval}}\}_{x=1}^{N_{\text{eval}}} . \quad (11)$$

探索为判断和巩固提供多样化的经验，而评估则验证由此产生的改进是否能迁移到更严格的条件。两个阶段使用不相交的随机种子，评估轨迹不会被添加到历史、记忆或训练数据中。

对于路径 p 和游戏 g ，循环 t 后的评估性能为

$$\bar{Y}_{t,g}^{(p)} = \frac{1}{N_{\text{eval}}} \sum_{x=1}^{N_{\text{eval}}} y_{t,g,x}^{(p)}, \quad (12)$$

相对于原始模型的改进为

$$\Delta_{t,g}^{(p)} = \bar{Y}_{t,g}^{(p)} - \bar{Y}_{0,g}. \quad (13)$$

本文报告最终评估分数和 $\Delta_{t,g}^{(p)}$ 。前者衡量最终能 t,g 力，后者衡量模型从自身交互历史中获益的有效程度。

4.5 自我改进路径

S³ Gym 评估了三条整合探索经验的路径。

历史上下文学习（History ICL）。带评分的轨迹被直接插入后续回合的上下文中：

$$Z_{t+1}^{(\text{History})} = C_{t+1}. \quad (14)$$

该路径保留了详细的交互证据，但受限于上下文长度。

摘要记忆（Summary Memory）。智能体接收的是整合后的摘要，而非完整轨迹：

$$Z_{t+1}^{(\text{Memory})} = M_{t+1}. \quad (15)$$

该路径评估模型能否从冗长的交互历史中抽象出可迁移的策略。

参数训练（Parameter Training）。探索轨迹与自我判断被转换为监督微调（Supervised Fine-Tuning）样本：

$$\mathcal{D}_t^{\text{SFT}} = \text{BuildSFT}(\mathcal{T}_t^{\text{exp}}, \mathcal{S}_t^{\text{exp}}), \quad (16)$$

随后进行参数更新

$$\theta_{t+1} = \text{SFT}(\theta_t, \mathcal{D}_t^{\text{SFT}}). \quad (17)$$

高分动作被保留为正样本，低评分动作被过滤或替换为修正后的动作。训练被视为辅助路径，因为它引入了上下文级改进中不存在的优化变量。

所有路径均从同一基础模型出发，并使用相同的探索种子、评估种子、交互预算和解码设置。

5 实验

5.1 基准游戏

S³ Gym 包含七个基于文本的游戏，涵盖互补的交互推理形式，包括潜在规则归纳、约束满足、数值变换、空间规划、长时程控制、资源分配和多智能体策略。每个游戏提供探索配置和评估配置。探索配置通常更宽松，而评估配置引入更严格的终止条件、更大的搜索空间或更密集的动态。表 2 总结了实验中使用的代码级配置。

游戏	探索	评估	最大步数
Chess	15 × 15 board, 12 pieces, and a 7-step history.	Same rules as exploration but 22 pieces.	5
Minesweeper	9 × 9 board with 10–20 mines and two extra lives.	The first mine hit terminates the episode.	64
Nullify	Expressions generated with 3–7 construction steps.	Expressions generated with 5–10 construction steps.	50
Plants-vs-Zombies	3 × 7 battlefield.	5 × 6 battlefield.	64
Snake	Collisions and invalid actions are treated as no-ops.	Collisions and invalid actions terminate the episode.	64
Tetris	10 × 10 board.	8 × 8 board.	64
Trust Evolution	Opponent sampling is biased toward easier strategies.	Opponent strategies are sampled uniformly.	10

表 2 七个 S³ Gym 游戏的探索与评估配置。探索通常使用更宽松的规则，而评估使用更严格或更困难的配置。

5.2 实验设置

模型。我们评估了七个具有代表性的专有大语言模型，其七个游戏结果均完整：GPT-4o、GPT-4.1、o3-mini、Gemini-2.5-Flash、Gemini-2.5-Pro、GPT-5.5 和 Gemini-3.5-Flash。检查点未完成的模型被排除在主表之外，以避免基于不同游戏覆盖范围的比较。

上下文级路径。我们比较两种非参数自改进路径。历史上下文学习（History ICL）直接将交互历史附加到上下文中，而摘要记忆（Summary Memory）将先前的轨迹压缩为模型生成的规则、错误和未来方向。前者保留了详细的证据，但需要更大的上下文预算；后者施加了信息瓶颈，因此更强烈地依赖于自我判断和摘要的质量。

Setting	Value
History-ICL input limit	100,000 tokens
Output generation cap	14,000 new tokens per response
Controller interaction cap	64 steps per episode
Exploration budget	30 exploration episodes
Evaluation interval	Every 3 exploration episodes
Evaluation set size	3 strict-mode episodes per checkpoint
Evaluation checkpoints	$x \in \{0, 3, 6, \dots, 30\}$

表 3 主要上下文级实验的运行配置。小于 64 的游戏特定视野会覆盖控制器级别的步数上限。

度量。设 $y_{m,p,g,k}$ 表示模型 m 、路径 p 和游戏 g 在检查点 x_k 处的评估分数。我们为每个模型-游戏对报告三个互补的度量：

$$\text{Avg}_{m,p,g} = \frac{1}{K+1} \sum_{k=0}^K y_{m,p,g,k}, \quad (18)$$

$$\text{Max}_{m,p,g} = \max_{0 \leq k \leq K} y_{m,p,g,k}, \quad (19)$$

$$\text{AUC}_{m,p,g}^+ = \int_0^{30} \max(0, y_{m,p,g}(x) - y_{m,p,g,0}) dx. \quad (20)$$

连续曲线 $y(x)$ 在曲线下面积 + 中通过相邻检查点之间的分段线性插值获得。平均值衡量整个探索过程中的整体性能；最大值捕捉观察到的最佳能力，而曲线下面积 + 衡量相对于初始评估分数的持续改进。由于游戏使用不同的分数尺度，原始曲线下面积 + 值应在同一游戏内进行比较，而不是跨游戏直接比较。

5.3 主要结果

表 4 报告了所有完整模型在历史上下文学习和摘要记忆下的平均值、最大值和曲线下面积 + 。粗体值表示在相应路径和游戏中每个度量的最佳结果。

整体上下文级性能。结果表明，自改进潜力由基础模型、经验路径和环境共同决定。在历史上下文学习下，双子座-3.5-闪电（Gemini-3.5-Flash）在象棋上获得最强结果，并在贪吃蛇和信任演化上获得最大的曲线下面积，而 GPT-5.5 在更脆弱的扫雷、归零和俄罗斯方块任务上领先，并实现了最大的植物大战僵尸曲线下面积。在摘要记忆下，GPT-5.5 在象棋、归零和俄罗斯方块上变得最强，并在贪吃蛇上达到最高的平均分和最高分。双子座-3.5-闪电在植物大战僵尸和信任演化的平均性能上保持最强。这些结果表明，仅靠高基础能力并不能决定自改进；模型在检索、压缩和应用自身经验的有效性上也存在差异。

历史上下文学习与摘要记忆。摘要记忆并非在所有情况下都优于原始历史。对于难以直接利用长轨迹的模型，它能带来显著提升。例如，双子座 2.5 闪电（Gemini-2.5-Flash）在扫雷任务中的曲线下面积（AUC）+ 从上下文学习下的 0.000 提升至使用摘要记忆后的 7.794，其在植物大战僵尸任务中的曲线下面积 + 从 24.402 提升至 238.501。GPT-5.5 在国际象棋任务中的曲线下面积 + 同样从 0.474 提升至 16.840。然而，摘要可能丢弃特定状态的证据或保留错误判断。GPT-5.5 在植物大战僵尸任务中的曲线下面积 + 从 548.499 下降至 33.219，而双子座 3.5 闪电在国际象棋任务中从 12.280 下降至 0.000，并且

表 4 七个 S₃ Gym 游戏的主要结果。对于每个模型-游戏组合，平均值为所有可用评估检查点的平均得分，最大值为最高得分，曲线下面积 + 为初始得分基线以上的正面积。所有度量指标均为越高越好。明确标记的并发失败已被排除。

(a) Average score (Avg.)							
Model	Chess	Minesweeper	Nullify	Tetris	Snake	PvZ	TrustEvo
ICL (History Context)							
GPT-4o	0.007	0.044	0.030	0.030	0.212	11.636	8.242
GPT-4.1	0.010	0.042	0.030	0.152	0.515	16.909	7.371
o3-mini	0.012	0.222	0.030	1.273	4.000	26.333	9.242
Gemini-2.5-Flash	0.009	0.063	0.030	0.121	1.424	20.697	9.008
Gemini-2.5-Pro	0.025	0.421	0.212	0.273	1.576	28.242	14.500
GPT-5.5	0.017	0.604	0.545	4.242	9.061	44.033	7.848
Gemini-3.5-Flash	0.547	0.482	0.394	1.970	8.455	44.697	17.242
Summary Memory							
GPT-4o	0.005	0.013	0.000	0.030	0.303	11.818	13.947
GPT-4.1	0.010	0.012	0.030	0.182	0.545	14.727	13.371
o3-mini	0.006	0.425	0.061	1.788	6.727	26.424	7.174
Gemini-2.5-Flash	0.074	0.438	0.091	0.152	1.364	21.939	8.720
Gemini-2.5-Pro	0.110	0.466	0.182	0.545	1.273	19.667	9.568
GPT-5.5	0.533	0.742	0.576	3.515	10.697	33.500	8.697
Gemini-3.5-Flash	0.208	0.603	0.455	1.091	10.242	43.879	14.394
(b) Maximum score (Max.)							
Model	Chess	Minesweeper	Nullify	Tetris	Snake	PvZ	TrustEvo
ICL (History Context)							
GPT-4o	0.018	0.131	0.333	0.333	0.667	17.667	11.500
GPT-4.1	0.018	0.189	0.333	0.667	1.667	21.667	10.250
o3-mini	0.022	0.500	0.333	2.667	9.667	29.333	13.250
Gemini-2.5-Flash	0.022	0.670	0.333	0.333	2.333	29.000	13.583
Gemini-2.5-Pro	0.089	0.659	0.667	1.000	4.333	33.000	19.000
GPT-5.5	0.067	0.974	1.000	11.333	11.333	48.000	23.000
Gemini-3.5-Flash	0.742	0.705	1.000	3.000	13.000	62.667	21.000
Summary Memory							
GPT-4o	0.018	0.033	0.000	0.333	1.000	20.333	19.333
GPT-4.1	0.031	0.083	0.333	0.667	1.000	19.333	15.917
o3-mini	0.018	0.608	0.333	4.000	10.333	29.000	10.083
Gemini-2.5-Flash	0.147	0.665	0.333	0.667	2.333	26.333	13.750
Gemini-2.5-Pro	0.258	0.974	0.333	2.000	3.000	26.667	14.333
GPT-5.5	0.773	1.000	1.000	6.667	14.000	41.333	17.000
Gemini-3.5-Flash	0.524	0.961	1.000	2.333	12.000	51.667	20.333
(c) Positive area above baseline (AUC ⁺)							
Model	Chess	Minesweeper	Nullify	Tetris	Snake	PvZ	TrustEvo
ICL (History Context)							
GPT-4o	0.233	1.437	0.500	1.000	0.000	0.052	3.443
GPT-4.1	0.186	1.329	1.000	0.000	8.501	129.000	5.966
o3-mini	0.004	0.000	1.000	0.000	0.000	5.383	62.638
Gemini-2.5-Flash	0.017	0.000	1.000	0.000	7.317	24.402	6.251
Gemini-2.5-Pro	0.000	3.539	1.500	0.000	0.000	60.416	161.876
GPT-5.5	0.474	0.959	1.000	96.251	0.163	548.499	164.411
Gemini-3.5-Flash	12.280	1.217	0.417	34.000	37.387	161.945	200.000
Summary Memory							
GPT-4o	0.007	0.046	0.000	0.500	1.917	12.778	10.350
GPT-4.1	0.177	0.046	1.000	0.000	0.000	75.462	154.876
o3-mini	0.079	0.001	2.000	28.598	0.300	40.425	30.521
Gemini-2.5-Flash	2.147	7.794	3.000	0.500	0.000	238.501	18.443
Gemini-2.5-Pro	0.573	0.721	0.000	10.084	11.321	11.872	10.624
GPT-5.5	16.840	2.951	7.418	50.801	0.000	33.219	240.500
Gemini-3.5-Flash	0.000	0.793	5.584	15.808	0.750	123.535	55.095

注：曲线下面积 (AUC) + 是在检查点 $x \in \{0, 3, \dots, 30\}$ 上通过分段线性梯形积分 $\int_0^{30} \max(0, y(x) - y_0) dx$ 计算得到的。均值和最大值排除了明确标记为无效的并发点；对于曲线下面积 +，相邻的有效检查点被连接起来。在任何游戏中结果未完成的模型均被省略。

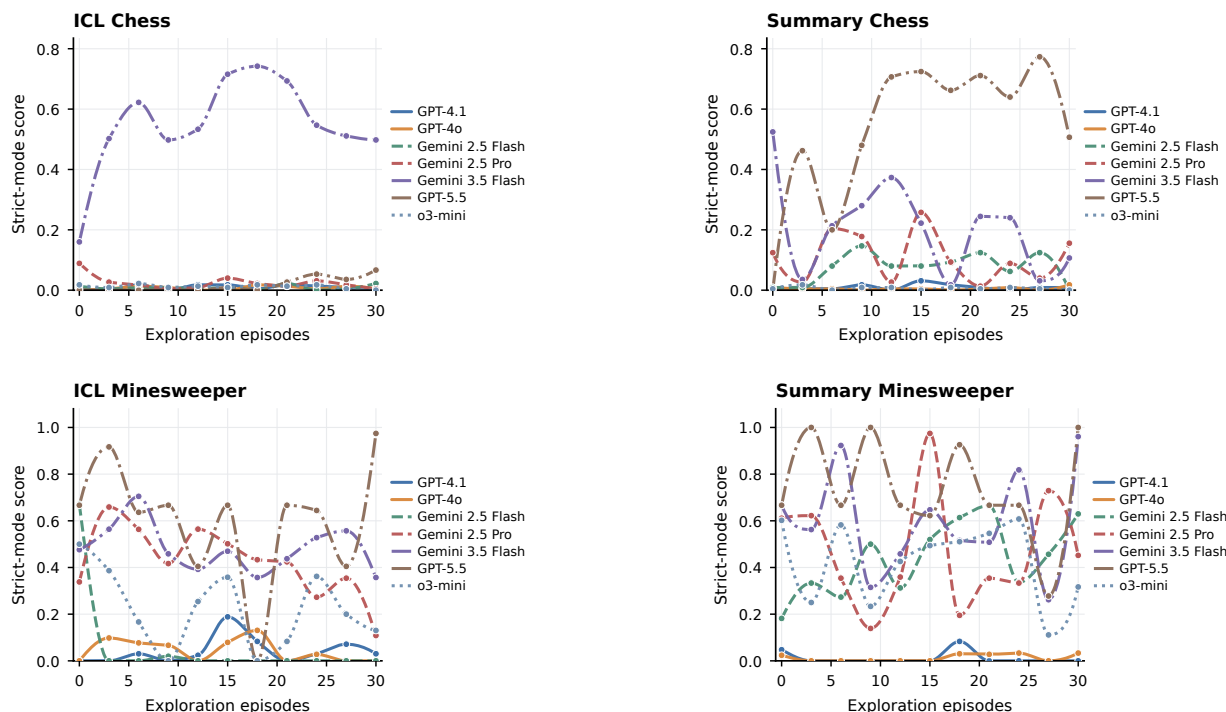


图 3 国际象棋与扫雷的配对历史-上下文学习与摘要-记忆轨迹。横轴表示探索回合数，纵轴表示严格模式评估性能。在每个游戏内，两个面板使用相同的纵轴刻度。

从 200.000 到 55.095 关于信任演化。因此，摘要记忆应被视为一种选择性压缩机制，而非对历史上下文学习的无条件改进。

跨游戏一致性。七款游戏揭示了不同的自我改进机制。国际象棋、扫雷和归零游戏具有稀疏或高度离散的分值，因此单个成功回合就能强烈影响最大值，而仅产生较小的平均值或曲线下面积。俄罗斯方块、贪吃蛇、植物大战僵尸和信任演化提供更密集的序列反馈，并更清晰地展现持续适应能力的差异。因此，三种度量是互补的：最大值识别偶发的能力突破，平均值反映整体性能，而曲线下面积 + 区分持续改进与孤立峰值。由于各游戏的分数尺度差异显著，跨游戏一致性应通过游戏内秩或改进的游戏数量来评估，而非对原始曲线下面积幅度取平均。

6 分析

6.1 研究问题一：参数训练能否实现自我改进？

研究问题一考察智能体能否通过参数更新内化其交互经验，并在推理时无需访问历史轨迹或外部记忆即可实现后续改进。与上下文层面的适应相比，参数训练提供了更为持久的改进途径，但也伴随着更大的风险：带有噪声或判断错误的轨迹可能被固化到模型中，从而覆盖先前有效的行为。

我们在 20 个连续检查点上评估了通义千问 3-8B 模型，其中第 0 轮代表原始模型，第 1 至 19 轮代表使用探索轨迹更新后的模型。在每个检查点上，模型均在全部七个 S₃ Gym 游戏的严格配置下进行评估，且不使用历史上下文学习或摘要记忆。我们报告训练后的平均分与最高分、最终变化量 $\Delta_{\text{Final}} = y_{19} - y_0$ 以及正向

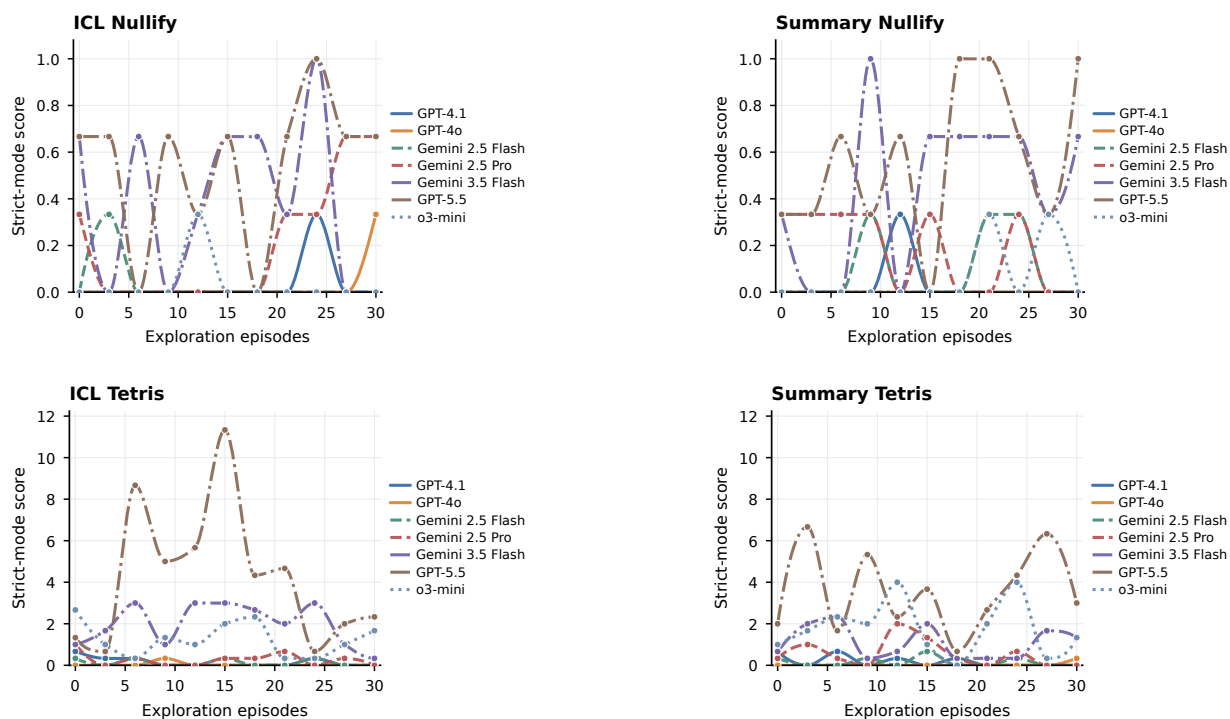


图 4 消除与俄罗斯方块在历史上下文学习和摘要记忆下的成对轨迹。模型颜色与线型在所有子图中保持一致。

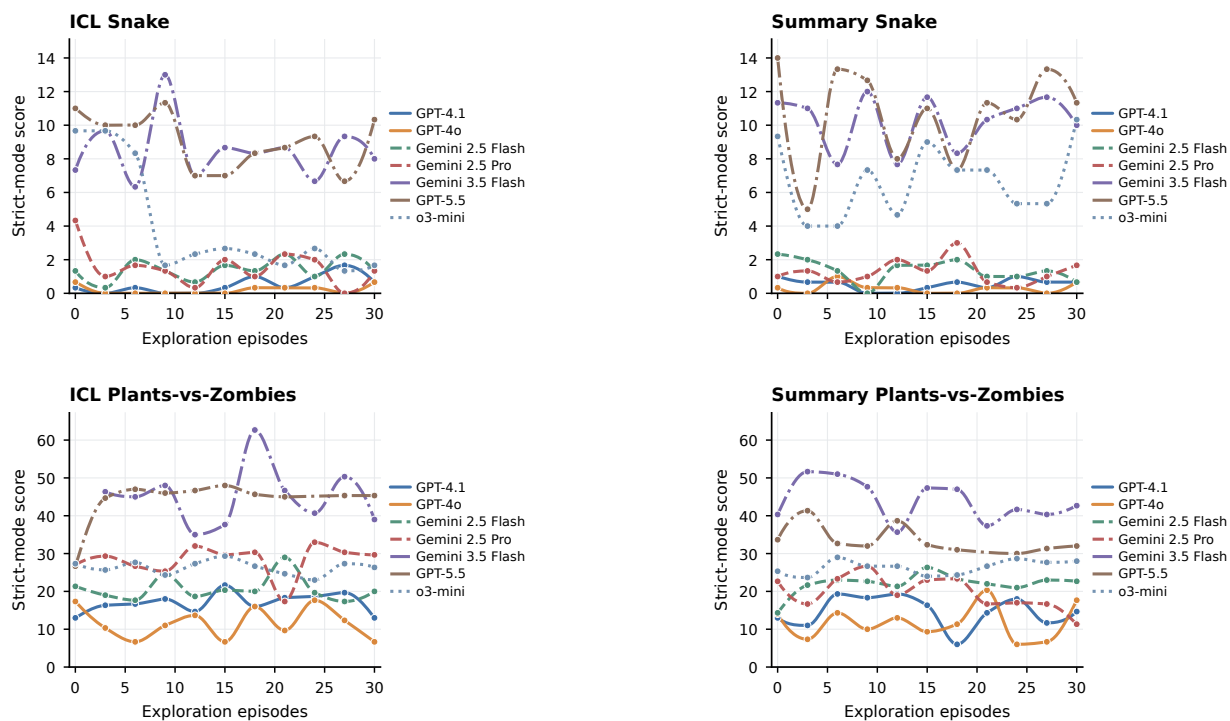


图 5 贪吃蛇与植物大战僵尸在历史上下文学习和摘要记忆下的成对轨迹。在每个游戏内，两种途径使用相同的纵轴刻度。

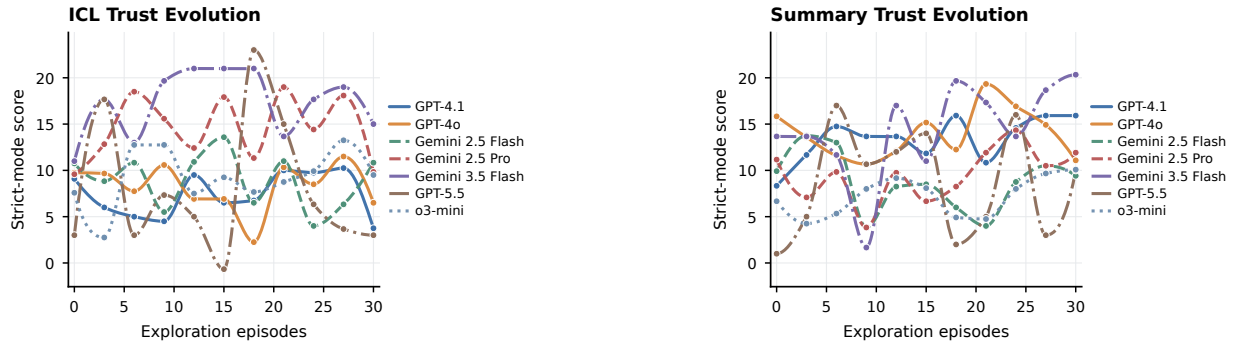


图 6 信任演化在历史上下文学习和摘要记忆下的成对轨迹。

初始基线以上的面积,

$$\text{AUC}^+ = \int_0^{19} \max(0, y(e) - y_0) de,$$

其中 $y(e)$ 通过对相邻检查点进行线性插值获得。

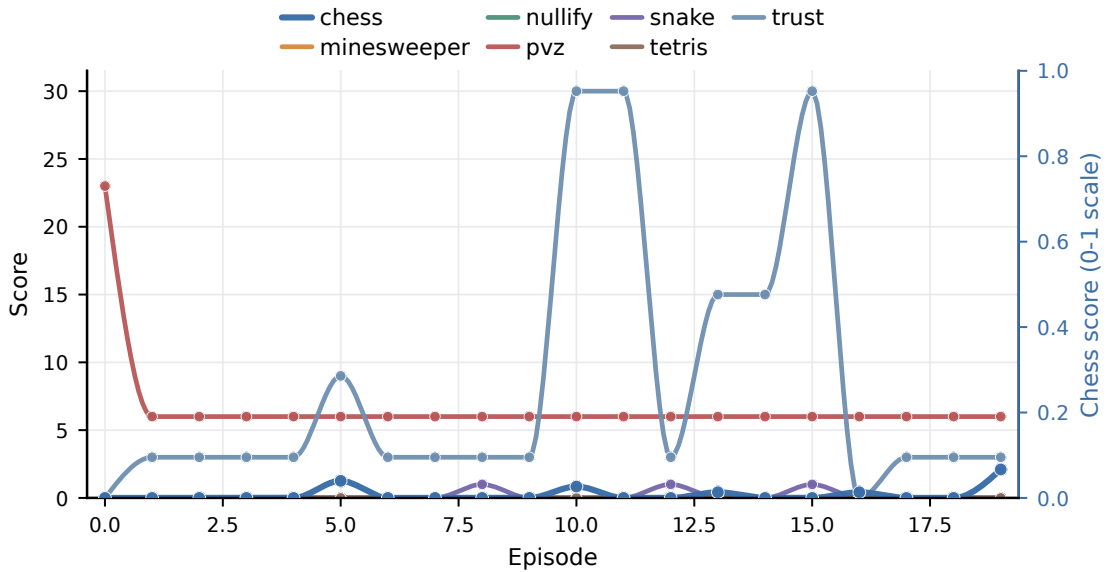


图 7 通义千问 3-8B 在 20 个训练检查点上的严格模式评估性能。第 0 轮表示原始模型。由于各游戏采用不同的分数尺度，每条轨迹应相对于其自身的初始性能进行解读。

- **训练在信任演化上带来了显著的自我提升。**分数从 0 升至最高 30，并在 19 个更新检查点中的 18 个上保持在初始基线之上。训练后平均分达到 8.684，曲线下面积 + 达到 163.5，这为交互衍生策略能够内化到模型参数中提供了最清晰的证据。

- **在国际象棋和贪吃蛇上的收益有限或短暂。**国际象棋在大多数检查点仍为零，但在最终检查点达到 0.0667，表明存在稀疏但正向的增益。贪吃蛇在第 8、12 和 15 个训练轮次达到 1 分，但随后又回到零。因此，训练过程中偶尔会发现成功行为，但未能持续保持。

- **扫雷、消除与俄罗斯方块未显示出可测量的改进。**它们的评估分数在整个训练过程中保持为零。一种可能的解释是，探索产生的成功次数太少

轨迹以引导有效的监督信号，尽管验证这一假设需要轨迹层面的分析。

- **训练在植物大战僵尸上产生了明显的负迁移。** 初始得分为 23，而每次更新后的检查点得分均为 6。这种持续的退化表明参数更新可能覆盖了先前有效的行为。潜在的原因包括对探索轨迹的过拟合、宽松探索与严格评估之间的不匹配，或不可靠的自我评判信号。

- 总体而言，参数训练能够实现自我改进，但在不同任务上表现不一致。Qwen3-8B 在信任演化任务上显著提升，在国际象棋上提升微弱，在贪吃蛇上间歇性提升，而在三个游戏上无改进，在植物大战僵尸上严重退化。因此，有效的训练级自我改进不仅取决于更新参数的能力，还取决于被内化经验的质量、多样性和正确性。

6.2 研究问题 2：智能体自我评判是否可靠？

自我评判是 S3 Gym 的核心组成部分，因为模型生成的得分 s_i 与每个转移一起存储，并随后影响轨迹选择、摘要构建和训练数据生成。如果 s_i 与环境信号不一致，智能体可能会将有害动作保留为有用经验，或丢弃本应保留的转移。因此，我们将自我评判作为一项独立能力进行评估，并考察评判质量是否能预测后续改进。

步骤级可靠性。对于每个转移，我们将模型生成的得分 s_i 与环境奖励 r_i 进行比较。我们首先测量事件一致性，该指标表示模型与环境在是否存在积极结果上是否一致：

$$\text{Agree} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\mathbf{1}(s_i > 0) = \mathbf{1}(r_i > 0)]. \quad (21)$$

由于不同游戏的奖励尺度不同，我们额外报告归一化平均绝对误差：

$$\text{NMAE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{s_i - s_{\min}}{s_{\max} - s_{\min}} - \frac{r_i - r_{\min}}{r_{\max} - r_{\min}} \right|. \quad (22)$$

我们还测量过度自信 $\Pr(s_i > 0, r_i \leq 0)$ 和自信不足 $\Pr(s_i \leq 0, r_i > 0)$ 。分析涵盖七个目标模型和游戏，在直接历史记录和摘要记忆两种条件下进行，共包含 98 次运行和 116,117 个转移。

Game	Agreement	NMAE	Over-conf.	Under-conf.
Chess	0.496	0.141	0.365	0.139
Minesweeper	0.820	0.524	0.062	0.118
Nullify	0.827	0.056	0.170	0.004
PvZ	0.881	0.882	0.032	0.087
Snake	0.879	0.121	0.110	0.011
Tetris	0.846	0.041	0.151	0.002
Trust	0.665	0.392	0.291	0.044

表 5 模型自我评判的步骤级可靠性。 一致性衡量转移是否获得积极得分的二元一致性。NMAE 衡量游戏级最小-最大归一化后的得分校准。

表 5 显示，自我评判 (Self-Judging) 仅部分可靠。植物大战僵尸 (PvZ)、贪吃蛇 (Snake)、俄罗斯方块 (Tetris)、扫雷 (Minesweeper) 和消除 (Nullify) 表现出较高的事件一致性，但这一结果部分归因于零奖励转移所占比例较大。

归一化平均绝对误差（NMAE）揭示了模型判断的幅度是否也得到良好校准。俄罗斯方块和消除实现了较低的归一化平均绝对误差，而植物大战僵尸尽管二元一致性较高，却表现出最大的校准误差。因此，模型能够识别某个动作在局部是否看似有用，同时却大幅误估其价值。

国际象棋（Chess）和信任（Trust）是最明显的失败案例。国际象棋的事件一致性接近随机水平，且过度自信率最高。一个预测可能在语法上完整，却错误放置了多个棋子，导致模型对其响应的置信度与环境得分产生偏差。信任同样表现出较弱的一致性、较高的归一化平均绝对误差以及显著的过度自信。这些结果表明，当奖励依赖于隐藏动态、对手策略或精确的多对象状态预测，而非立即可观察的局部后果时，自我评判的可靠性会降低。

评判-改进耦合。可靠的局部评判只有在智能体能够将其转化为后续更好的行为时才有用。为检验这一关系，我们将探索轨迹划分为相邻评估检查点之间的块 B_t 。对于每个块，我们计算事件一致性 A_t 、归一化校准误差 E_t 以及后续评估中的归一化得分变化：

$$g_t = \frac{y_t - y_{t-3}}{\max_{g,\tau} y_{g,\tau} - \min_{g,\tau} y_{g,\tau}}. \quad (23)$$

然后我们测量 $\rho(A_t, g_t)$ 和 $\rho(-E_t, g_t)$ ，以及最高和最低评判质量三分位组之间平均得分增益的差异。

游戏	$\rho(A, g)$	$\rho(-E, g)$	Gap_A	Gap_E
All	-0.010	-0.018	-0.004	-0.016
Chess	-0.014	-0.058	0.028	-0.039
Minesweeper	0.071	0.006	0.069	0.005
Nullify	-0.032	-0.082	-0.065	-0.058
PvZ	-0.038	-0.023	0.014	0.010
Snake	0.022	0.022	0.009	0.009
Tetris	-0.048	-0.021	0.010	0.004
Trust	-0.014	-0.003	-0.001	0.006

表 6 分块评判-改进耦合。 A 为事件一致性， E 为归一化校准误差， g 为后续评估中归一化严格模式得分变化。差距度量比较最高和最低评判质量三分位组。

在整个基准上，耦合结果接近于零： $\rho(A, g) = -0.010$ 和 $\rho(-E, g) = -0.018$ 。相应的三分位差距也很小且略为负值。扫雷在事件一致性与后续得分增益之间表现出微弱的正相关关系，但其校准耦合仍然可以忽略不计。总体而言，准确的局部评分并不能可靠地在下一次评估检查点产生改进。

运行层面的分析得出相同结论。为量化持续改进并兼顾初始性能差异，本文采用归一化超基线面积（Normalized Above-Baseline Area, NABA），其定义为

$$\text{NABA} = \frac{\text{AUC}^+}{\max(y_0, \epsilon)},$$

其中 y_0 为初始评估得分， ϵ 表示当 $y_0 = 0$ 时的首个非零得分尺度。事件一致性在皮尔逊（Pearson）和斯皮尔曼（Spearman）测度下均与 NABA 仅呈弱负相关（ $r = -0.23$ 和 $\rho = -0.11$ ），而正奖励率和正自我得分率仅呈弱正相关。这些结果表明，仅靠准确的局部自我判断不足以实现自我改进：智能体还必须将反馈转化为可复用的状态抽象、决策规则和探索策略。

6.3 研究问题三：得分条件化摘要能否产生有效的迭代方向？

摘要记忆 (Summary Memory) 对直接历史复用提出了更强要求。模型不能仅以先前轨迹为条件，还必须识别哪些低得分行为应被抑制，哪些高得分行为应被保留，以及如何将这些观察转化为未来回合的可执行策略。本文通过将得分条件化摘要与直接历史上下文学习 (In-Context Learning, ICL) 进行比较，并检查具有代表性的成功与失败摘要，来评估这一能力。

摘要记忆与直接历史对比。对于每个模型-游戏对，本文计算 ΔNABA 为摘要记忆 NABA 减去直接历史 NABA。正值表明摘要化比保留原始交互历史产生了更有效的改进轨迹。

Game	Pairs	Summary wins	History wins	Ties	Mean ΔNABA
Chess	7	3	3	1	3.37
Minesweeper	7	3	4	0	-1.12
Nullify	7	4	2	1	2.79
PvZ	7	3	4	0	-2.11
Snake	7	3	4	0	-0.81
Tetris	7	5	1	1	6.20
Trust	7	4	3	0	8.43

表 7 摘要记忆与直接历史上下文学习对比。每对在两种历史组织策略下评估相同的模型和游戏。

摘要记忆在消除 (Nullify)、俄罗斯方块 (Tetris) 和信任 (Trust) 上提高了平均 NABA。这些环境允许紧凑且可复用的抽象，例如向后规划以实现精确抵消、保持稳定的棋盘表面，或根据推断的对手策略调整行为。在此类情况下，摘要移除了冗余的轨迹细节，同时保留了与未来决策最相关的信息。

相比之下，直接历史在扫雷、植物大战僵尸和贪吃蛇中的平均表现更好。这些游戏高度依赖局部棋盘配置、路线时机、危险位置以及部分可观测的风险。摘要可能保留了一个合理的高层目标，却丢弃了正确执行该目标所需的状态相关细节。因此，摘要记忆在经验可以被压缩为稳定策略规则时最为有效，而在成功取决于重建精确当前状态的情况下则不然。

轨迹层面的证据。以下案例说明了何时基于分数的摘要能够成为可执行的迭代方向，以及何时它们仍然过于笼统而无法改善行为。

成功的摘要

GPT-5.5 / Trust

摘要方向。 Always defect when the observed payoff structure consistently favors defection; avoid unnecessary reciprocal exploration.

轨迹证据。 At the first decision, the agent selected `cheat` and received an environment reward of 3.0.

Outcome. $\Delta\text{NABA} = +66.89$.

Takeaway. The summary translates payoff feedback into an explicit opponent-conditioned policy.

o3-mini / Tetris

摘要方向。 Jointly inspect the current and next blocks, test alternative rotations, prioritize line-clearing placements, and maintain a low surface without isolated holes.

轨迹证据。 In episode 10, the agent cleared four lines. It selected `8 0` twice, with both its self-score and the environment reward equal to 1.0 on each move.

Outcome. $\Delta\text{NABA} = +14.42$.

Takeaway. The summary converts successful line clears into an executable look-ahead rule.

Gemini-2.5-Flash / Minesweeper

摘要方向。 Apply local number constraints: expand zero regions, flag forced mines, and uncover cells that are provably safe.

轨迹证据。 The agent selected `flag (2,2)` on a partially revealed board and received a reward of 0.0625.

Outcome. $\Delta\text{NABA} = +25.49$.

Takeaway. Summaries help when they express local reasoning as executable constraint rules.

Gemini-3.5-Flash / Nullify

Summary direction. Plan backward to create exact opposites, use floor or ceiling operations to remove decimals, and track index changes after every merge.

轨迹证据。 With terminal units `-19` and `19`, the agent selected `0 1` and received a reward of 1.0.

Outcome. $\Delta\text{NABA} = +8.50$.

Takeaway. The summary succeeds because it specifies a concrete arithmetic planning procedure.

Failure summaries

GPT-5.5 / PvZ

摘要方向。 Prioritize sun production, early defense, Wall-nut stalling, and coverage of threatened lanes.

轨迹证据。 The agent opened with `X 1 0` and received a reward of 1.0, but its later strict score fell from 38.67 to 32.33.

Outcome. $\Delta\text{NABA} = -2.56$.

Takeaway. The advice is reasonable but too coarse to capture lane timing, current zombie positions, and the available sun budget.

Gemini-3.5-Flash / Chess

摘要方向。 Track the movement rule of every piece and verify all coordinates before submitting the answer.

Trajectory evidence. In a low-scoring episode, the model produced `END` and received zero total reward despite having generated a detailed rule summary.

Outcome. $\Delta\text{NABA} = -33.27$.

Takeaway. A high-level reminder does not resolve the exact multi-piece localization problem.

GPT-4.1 / Minesweeper

摘要方向。 Begin from corners, expand zero regions, flag only certain mines, and stop when no safe progress is available.

轨迹证据。 The agent selected `uncover (8,0)` after exhausting its available lives, and the subsequent checkpoint score fell from 0.0606 to 0.

Outcome. $\Delta\text{NABA} = -22.09$.

Takeaway. Generic heuristics do not substitute for exact constraint propagation.

GPT-4o / Snake

摘要方向。 Move toward food while avoiding walls, obstacles, the snake body, and direct reversals.

轨迹证据。 Given a board containing food, obstacles, and its own body, the agent selected `DOWN`; it received no reward, and the strict score fell from 1.0 to 0.

Outcome. $\Delta\text{NABA} = -22.00$.

Takeaway. The summary omits the precise geometry required for safe navigation.

综合来看，这些案例揭示了基于摘要的改进有效所需的两个条件。首先，分数必须识别出有意义且可复用的失败模式，而不仅仅是表明回合结束时的结果不佳。其次，摘要必须将该信号转化为足够精确的动作规则，以便在新状态下执行。因此，摘要记忆是一种选择性的改进机制：当经验允许紧凑的因果抽象时它是有效的，但它不能普遍替代对状态丰富的交互历史的直接访问。

7 结论

S3GYM 将自我改进从一个宽泛的主张转变为可衡量的智能体能力。通过将宽松的探索与更严格的评估分离，并支持轨迹在上下文层面和训练层面的重用，该基准测试检验了大语言模型能否将交互历史转化为未来的性能。迄今为止的证据以一种有用的方式呈现出混合性。摘要记忆在紧凑规则可以跨回合迁移的游戏有所帮助，而直接历史在细节至至关重要的反应式控制游戏中往往表现更好。在当前实现中，基于自生成轨迹的训练仍然不稳定。这些发现表明，未来的自我改进智能体需要更好的判断校准、记忆选择和轨迹过滤，而不仅仅是更多的交互数据。

8 贡献

核心贡献者 石佳俊、陶思远、吴宇浩、王泽轩、张静远

贡献者 刘佳恒、雷新平、张新荣、方思远、谭哲文、蔡天乐、方俊豪、黄佳萌、王岳阳、刘金凯、张宇轩

通讯作者 杨健、李周军、严深、黄文浩、张戈

参考文献

- [1] Emre Can Acikgoz, Cheng Qian, Heng Ji, Dilek Hakkani-Tür, and Gokhan Tur. Self-improving llm agents at test-time. arXiv preprint arXiv:2510.07841, 2025. URL <https://arxiv.org/abs/2510.07841>.
- [2] Chris Argyris and Donald A. Schön. Organizational Learning: A Theory of Action Perspective. Addison-Wesley, Reading, MA, 1978. ISBN 9780201001747.
- [3] Aditya Challapally, Chris Pease, Ramesh Raskar, and Pradyumna Chari. The GenAI divide: State of AI in business 2025. https://www.artificialintelligence-news.com/wp-content/uploads/2025/08/ai_report_2025.pdf, 2025. Industry report.
- [4] Yizhe Chi, Wenyi Li, Deyao Hong, Xiaoqi Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. Ai4ai-bench: Benchmarking llm agents in algorithmic design for recursive self-improvement. arXiv preprint arXiv:2608.20318, 2026. URL <https://arxiv.org/abs/2608.20318>.
- [5] Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Ruo Yu Tao, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. TextWorld: A learning environment for text-based games. arXiv preprint arXiv:1806.11532, 2018. URL <https://arxiv.org/abs/1806.11532>.
- [6] Bo Deng, Kang Zhou, Lifan Guo, Chongyang Tao, Xuanren Chen, Chenggang Xie, Renzhao Liang, Feng Chen, and Chi Zhang. Finevo-bench: A longitudinal benchmark for self-evolving agents in professional financial workflows. arXiv preprint arXiv:2608.06144, 2026. URL <https://arxiv.org/abs/2608.06144>.
- [7] Zihao Deng, Yining Zhu, Leiming Wang, Junbo Wang, and Jingfei Lu. Tree-of-experience: Hierarchical experience management for self-evolving agents. arXiv preprint arXiv:2608.09044, 2026. URL <https://arxiv.org/abs/2608.09044>.
- [8] John Dewey. How We Think: A Restatement of the Relation of Reflective Thinking to the Educative Process. D. C. Heath and Company, Boston, MA, 2 edition, 1933.
- [9] Zi-Yi Dou, Cheng-Fu Yang, Xueqing Wu, Kai-Wei Chang, and Nanyun Peng. Re-ReST: Reflection-reinforced self-training for language agents. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 15394–15411, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.861. URL <https://aclanthology.org/2024.emnlp-main.861/>.
- [10] Tianyi Guan, Yiding Wang, Haotong Yang, Siyuan Cao, Shirui Liu, Yi Hu, Jiaqi Li, and Muhan Zhang. Continualskillbench: Can llm agents truly evolve their capabilities? arXiv preprint arXiv:2608.03874, 2026. URL <https://arxiv.org/abs/2608.03874>.
- [11] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (ReST) for language modeling. arXiv preprint arXiv:2308.08998, 2023. URL <https://arxiv.org/abs/2308.08998>.
- [12] Keyu He, Xuhui Zhou, and Maarten Sap. Social gym and spartan: Benchmarking and improving llm social reasoning via multi-agent game tournaments. arXiv preprint arXiv:2608.09128, 2026. URL <https://arxiv.org/abs/2608.09128>.

- [13] Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Shisong Chen, Jinghao Zhang, Tianjun Pan, Weijia Zhou, Jiaqing Liang, and Yanghua Xiao. Sea-eval: A benchmark for evaluating self-evolving agents beyond episodic assessment. *arXiv preprint arXiv:2604.08988*, 2026. URL <https://arxiv.org/abs/2604.08988>.
- [14] David A. Kolb. *Experiential Learning: Experience as the Source of Learning and Development*. Prentice-Hall, Englewood Cliffs, NJ, 1984. ISBN 9780132952613.
- [15] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- [16] Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL <https://arxiv.org/abs/2401.13178>.
- [17] James G. March. Exploration and exploitation in organizational learning. *Organization Science*, 2(1):71–87, 1991. doi: 10.1287/orsc.2.1.71.
- [18] Gergely Orosz. What are forward deployed engineers, and why are they so in demand? <https://newsletter.pragmaticengineer.com/p/forward-deployed-engineers>, 2025. The Pragmatic Engineer.
- [19] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025. URL <https://arxiv.org/abs/2509.25140>.
- [20] Palantir Technologies. A day in the life of a Palantir forward deployed software engineer. <https://blog.palantir.com/a-day-in-the-life-of-a-palantir-forward-deployed-software-engineer-45ef2de257b1>, 2020.
- [21] Karl R. Popper. *The Logic of Scientific Discovery*. Hutchinson, London, 1959.
- [22] Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. Posttrainbench: Can llm agents automate llm post-training? In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2603.08640>.
- [23] Donald A. Schön. *The Reflective Practitioner: How Professionals Think in Action*. Basic Books, New York, NY, 1983. ISBN 9780465068746.
- [24] Linfang Shang, Ming Xu, Yiding Sun, Tianle Xia, Lingxiang Hu, Lan Xu, and Ning Zheng. When self-evolution backfires: Pre-commit gating against skill contamination in llm agents. *arXiv preprint arXiv:2608.05810*, 2026. URL <https://arxiv.org/abs/2608.05810>.
- [25] Jiajun Shi, Jian Yang, Jiaheng Liu, Xingyuan Bu, Jiangjie Chen, Juntao Zhou, Kaijing Ma, Zhoufutu Wen, Bingli Wang, Yancheng He, Liang Song, Hualei Zhu, Shilong Li, Xingjian Wang, Wei Zhang, Ruibin Yuan, Yifan Yao, Wenjun Yang, Yunli Wang, Siyuan Fang, Siyu Yuan, Qianyu He, Xiangru Tang, Yingshui Tan, Wangchunshu Zhou, Zhaoxiang Zhang, Zhoujun Li, Wenhao Huang, and Ge Zhang. KORGym: A Dynamic Game Platform for LLM Reasoning Evaluation. *arXiv e-prints*, art. arXiv:2505.14552, May 2025. doi: 10.48550/arXiv.2505.14552.
- [26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- [27] Mohit Shridhar, Kingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *The Ninth International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=0IOX0YcCdTn>.
- [28] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=G0dksFayVq>.
- [29] The New Stack. Why OpenAI and Anthropic are hiring forward deployed engineer teams. <https://thenewstack.io/forward-deployed-engineers-ai/>, 2026.

- [30] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL <https://arxiv.org/abs/2305.16291>.
- [31] Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11279–11298, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.775. URL <https://aclanthology.org/2022.emnlp-main.775/>.
- [32] Wenxiao Wang, Priyatham Kattakinda, and Soheil Feizi. Do agent optimizers compound? a continual-learning evaluation on terminal-bench 2.0. *arXiv preprint arXiv:2607.14004*, 2026. URL <https://arxiv.org/abs/2607.14004>.
- [33] Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. Metaskill-evolve: Recursive self-improvement of llm agents via two-timescale meta-skill evolution. *arXiv preprint arXiv:2607.05297*, 2026. URL <https://arxiv.org/abs/2607.05297>.
- [34] Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, Yuxin Chen, Mengdi Wang, and Ling Yang. Past-bench: Benchmarking the foundations of recursive self-improvement in personal agents. *arXiv preprint arXiv:2608.04003*, 2026. URL <https://arxiv.org/abs/2608.04003>.
- [35] Qinyuan Ye, Yu Li, Yada Pruksachatkun, Jiaxin Zhang, and Chien-Sheng Wu. On the fragility of self-improving agents: Variance, task order, and underspecification. *arXiv preprint arXiv:2608.18066*, 2026. URL <https://arxiv.org/abs/2608.18066>.
- [36] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason E. Weston. Self-rewarding language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 57905–57923. PMLR, 2024. URL <https://proceedings.mlr.press/v235/yuan24d.html>.
- [37] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, volume 35, pages 15476–15488, 2022. URL <https://arxiv.org/abs/2203.14465>.
- [38] Haiyue Zhang. Credit without ground truth: Auditing step-level credit assignment in llm agents against executed replay. *arXiv preprint arXiv:2608.19760*, 2026. URL <https://arxiv.org/abs/2608.19760>.
- [39] Xiaoying Zhang, Zichen Liu, Yipeng Zhang, Xia Hu, and Wenqi Shao. Retroagent: From solving to evolving via retrospective dual intrinsic feedback. *arXiv preprint arXiv:2603.08561*, 2026. URL <https://arxiv.org/abs/2603.08561>.
- [40] Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. Seagym: An evaluation environment for self-evolving llm agents. *arXiv preprint arXiv:2606.17546*, 2026. URL <https://arxiv.org/abs/2606.17546>.
- [41] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623, 2023. URL <https://arxiv.org/abs/2306.05685>.

附录

A 游戏提示模板

本附录记录了主实验中七个游戏所使用的规范提示。这些模板转录自 `test/S3GYM/envs/*/prompt.py`。我们规范化了空白字符，并将特定于回合的值替换为尖括号占位符（例如 `<BOARD>`）；游戏规则、动作空间、坐标约定以及所需答案语法均遵循实现。在推理时，游戏特定的系统提示会与当前步骤提示拼接，对于基于记忆的智能体，还会拼接可用的提示消息。

A.1 共享交互协议

六个游戏前置了下面描述的共享基础指令，而信任演化则使用其自身的游戏特定系统指令。两种记忆条件都保留相同的游戏规则、动态状态、合法动作空间和自评判契约。它们仅在如何表示当前状态之前来自先前探索回合的经验方面有所不同。

Direct memory: raw-history in-context learning

直接记忆将先前回合中每个保留的步骤序列化为环境-智能体对，即环境：<状态> 后跟 大语言模型：答案：<动作>，得分：<自评分>。在动作时刻，模型接收以下组合：

```
Hint Messages:
<RAW HISTORY FROM PREVIOUS EPISODES>

Game history of current episode:
<RAW HISTORY FROM EARLIER STEPS IN THIS EPISODE>

<GAME SYSTEM PROMPT + CURRENT STEP PROMPT>
```

该实现完整保留当前回合历史和当前游戏提示，同时在上下文预算超出时截断先前回合历史的尾部。

Summary memory: score-conditioned strategy compression

在每个回合的第 0 步，摘要记忆首先对游戏规则和截断的先前回合历史进行一次单独的压缩调用：

```
Rule:
<GAME SYSTEM PROMPT>

History:
<TRUNCATED RAW HISTORY FROM PREVIOUS EPISODES>

Please summarize practical experience and recommendations ...
Tips: <BULLET-POINTED STRATEGY SUMMARY>
```

The extracted Tips are cached for the session and reused at later steps. The action-time input is then

```
Hint Messages:
<SUMMARIZED TIPS>

<GAME SYSTEM PROMPT>

Game history of current episode:
<RAW HISTORY FROM EARLIER STEPS IN THIS EPISODE>

<GAME SYSTEM PROMPT + CURRENT STEP PROMPT>
```

因此，仅跨回合经验被压缩；当前回合仍以原始状态-动作-得分历史形式保留。在已发布的实现中，游戏系统提示首先与缓存的摘要前缀一同出现，随后再次与当前步骤提示一同出现。

无论记忆条件如何，模型都必须报告一个动作及其自评的即时得分。共享的末行约定为

Shared response contract

Answer: <ACTION>, **Score:** <SELF-JUDGED SCORE>.
If the agent determines that the episode has terminated, it returns **Answer:** END, **Score:** 0.

自评得分旨在表示所提动作在既定游戏规则下获得的奖励，而非自由形式的置信度值。表 8 在完整模板呈现之前，总结了观测与动作接口。

Game	Main dynamic observation	Valid action form
Chess	$N \times N$ board with labeled pieces and empty cells	Coordinates for one or more labeled pieces
Minesweeper	Partially revealed grid with numbers, flags, and unknown cells	uncover, flag, or unflag
Nullify	Indexed list of signed values and arithmetic operators	Two unit indices
Tetris	Fixed-cell board and next-block rendering	Drop column and rotation angle
Snake	Grid containing walls, head, body, food, and obstacles	LEFT/RIGHT/UP/DOWN
Plants-vs-Zombies	Battlefield, sun budget, plants, zombies, and turn state	Zero or more plant placements
Trust Evolution	Interaction round and action/payoff history	collaborate or cheat

表 8 规范游戏提示所暴露的观测与动作接口。

A.2 状态推断与演绎

Chess-style state prediction

系统指令。 You are playing a step-by-step piece-position prediction game on an $N \times N$ board. Empty cells are denoted by ., pieces are denoted by letters, coordinates are zero-based, and every piece follows its own hidden movement rule. Predict the next position of as many pieces as possible from the observed board-state sequence.
动态状态。 Current Game State: followed by <BOARD>.
决策指令。 Predict the next-step location of each recoverable piece. Use the syntax A:(x,y), b:(x,y), c:(x,y), The agent may return END when it elects to stop.

Answer: A:(<ROW>,<COL>), b:(<ROW>,<COL>), ..., **Score:** <SELF-JUDGED SCORE>.

Minesweeper

系统指令。 You are playing Minesweeper on a <BOARD_ROWS> $\times$ <BOARD_COLS> board with <MINES> hidden mines. Unknown cells are ?, flags are F, revealed counts are 0-8, and revealed mines are X. Coordinates are zero-based. Win by uncovering every safe cell or correctly flagging every mine; the episode ends at <MAX_EPOCHS> or when a mine is uncovered, depending on the evaluation mode.
动态状态。 Current Game State: followed by the partially observed <BOARD>.
决策指令。 Choose exactly one of uncover (r,c), flag (r,c), or unflag (r,c). The score is the ratio of correctly flagged mines to the total number of mines.

Answer: uncover (<ROW>,<COL>), **Score:** <SELF-JUDGED SCORE>.

A.3 符号与空间规划

Nullify

系统指令。 The board contains indexed independent units. Signed numbers ($+n$ or $-n$) can be combined with another unit; available operators include multiplication, division, square root, square, reciprocal, floor, and ceiling. Combining equal-magnitude values with opposite signs produces zero and removes both units. The objective is to eliminate all units, yielding a final value of zero.

动态状态。 Current game board: followed by one index,content pair per unit in <INDEXED UNITS>. Unit indices start at zero and are updated after each combination.

决策指令。 elect the two indices to combine on the current turn. A successful complete cancellation receives score 1; otherwise the terminal score is 0.

Answer: <INDEX I> <INDEX J>, Score: <SELF-JUDGED SCORE>.

Tetris

系统指令。 You are playing simplified Tetris on a $<BOARD_WIDTH> \times <BOARD_HEIGHT>$ board. The board shows fixed blocks as * and empty cells as .; the next-block panel uses #. A placed block falls until obstructed, each completed row is cleared for one point, and the episode terminates when a falling block extends above the board.

动态状态。 nt Game State: <BOARD> and Next Block: <NEXT_BLOCK>.

决策指令。 Choose the one-based drop column of the block's leftmost cell and a clockwise rotation in {0, 90, 180, 270}. The objective is to maximize cleared rows.

Answer: <DROP COLUMN> <ROTATION ANGLE>, Score: <SELF-JUDGED SCORE>.

A.4 反应式控制与策略交互

Snake

系统指令。 You control a snake on a $<GRID_WIDTH> \times <GRID_HEIGHT>$ grid. Walls are #, the head is H, body cells are S, apples are A, and obstacles are X. Eating an apple increases both length and score by one. A wall, self, obstacle, or invalid-action collision terminates the game under strict rules. The snake cannot reverse direction and the episode is capped at <MAX_STEPS>.

动态状态。 Current Game State: followed by <BOARD> and the current movement direction.

决策指令。 Choose exactly one next movement from LEFT, RIGHT, UP, or DOWN.

Answer: <LEFT|RIGHT|UP|DOWN>, Score: <SELF-JUDGED SCORE>.

Plants-vs-Zombies

系统指令。 You are playing simplified Plants-vs-Zombies on a $<ROWS> \times <COLS>$ grid. Each turn gains at least 25 sun; zombies spawn every five turns and move left, becoming stronger over time. Available plants are Sunflower (X), Peashooter (W), Three-Line Shooter (S), Wall-nut (J), Torch Stump (H), and Fire Chili (F), each with the costs and effects specified in the system prompt. The score increases by one per survived turn; the game ends when a zombie reaches column -1 or after <MAX_STEPS>.

动态状态。 Current Game State: followed by <BATTLEFIELD>, including the turn, sun budget, plant layout, and zombie states.

决策指令。 Place zero or more plants for the next turn. Each placement is PlantType Row Col; multiple placements are separated by semicolons. Rows and columns are zero-based, and an empty action is allowed.

Answer: X <ROW> <COL>; W <ROW> <COL>, Score: <SELF-JUDGED SCORE>.

Trust Evolution

系统指令。 You play a fixed-length repeated interaction against an opponent with a stable within-episode strategy. The two actions are collaborate and cheat. Payoffs to the agent are +2 for mutual collaboration, -1 for collaborating against cheating, +3 for cheating against collaboration, and 0 for mutual cheating.

动态状态。 Current Game State: followed by <ROUND AND INTERACTION HISTORY> for an episode of <MAX_STEPS> rounds.

决策指令。 Infer the opponent's behavior from the observed interaction and select exactly one action for the next round.

Answer: <collaborate|cheat>, Score: <SELF-JUDGED SCORE>.